

✓ Tutorial 01: Spatial Analysis of Visium H&E Data

Goal: Explore the relationship between tissue morphology and gene expression using Squidpy.

Dataset: Coronal section of the Mouse Brain (10x Genomics).

Key Methods: Image feature extraction, neighborhood enrichment, and Moran's I spatial autocorrelation.

Javeria Butt

✓ 1. Imports

```
%matplotlib inline

import numpy as np
import pandas as pd
import anndata as ad
import scanpy as sc
import squidpy as sq
import warnings
warnings.filterwarnings('ignore')

sc.logging.print_header()
print(f"squidpy=={sq.__version__}")

squidpy==1.8.1
```

✓ 2. Load Data

The dataset is a pre-processed Visium slide of a mouse brain coronal section with pre-annotated clusters. It is downloaded automatically by `sq.datasets`.

```
img = sq.datasets.visium_hne_image()
adata = sq.datasets.visium_hne_adata()

print(adata)
print(f"\nClusters: {adata.obs['cluster'].nunique()}")
print(adata.obs['cluster'].value_counts())
```

```
INFO      Downloading visium_hne_image.tiff from
https://exampledata.scverse.org/squidpy/figshare/visium\_hne\_image.tiff
Downloading data from 'https://exampledata.scverse.org/squidpy/figshare/visium\_hne\_image.tiff' to file '/content/data/images/vis
100%

398M/398M

INFO      Downloading visium_hne_adata.h5ad from
https://exampledata.scverse.org/squidpy/figshare/visium\_hne\_adata.h5ad
Downloading data from 'https://exampledata.scverse.org/squidpy/figshare/visium\_hne\_adata.h5ad' to file '/content/data/anndata/vi
100%

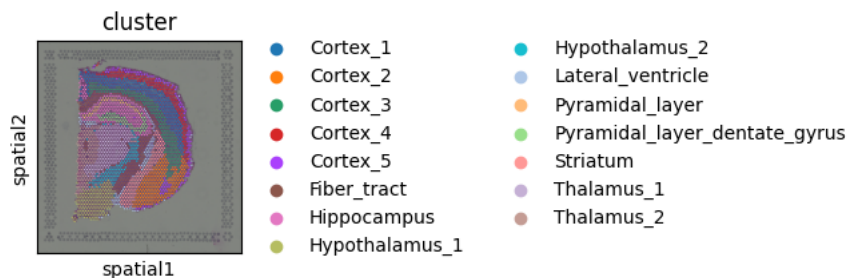
329M/329M

AnnData object with n_obs × n_vars = 2688 × 18078
  obs: 'in_tissue', 'array_row', 'array_col', 'n_genes_by_counts', 'log1p_n_genes_by_counts', 'total_counts', 'log1p_total_cou
  var: 'gene_ids', 'feature_types', 'genome', 'mt', 'n_cells_by_counts', 'mean_counts', 'log1p_mean_counts', 'pct_dropout_by_c
  uns: 'cluster_colors', 'hvg', 'leiden', 'leiden_colors', 'neighbors', 'pca', 'rank_genes_groups', 'spatial', 'umap'
  obsm: 'X_pca', 'X_umap', 'spatial'
  varm: 'PCs'
  obsp: 'connectivities', 'distances'

Clusters: 15
cluster
Cortex_1                284
Thalamus_1              261
Cortex_2                257
Cortex_3                244
Fiber_tract             226
Hippocampus             222
Hypothalamus_1          208
Thalamus_2              192
Cortex_4                164
Striatum                153
Hypothalamus_2          133
Cortex_5                129
Lateral_ventricle       105
Pyramidal_layer_dentate_gyrus 68
Pyramidal_layer         42
Name: count, dtype: int64
```

3. Spatial Visualization of Cluster Annotations

```
sq.pl.spatial_scatter(
    adata,
    color='cluster',
    save='../results/figures/01_visium_hne/spatial_clusters.png'
)
```



4. Image Features

Squidpy can extract image features from the high-resolution tissue image for each Visium spot. This allows us to incorporate tissue morphology information alongside gene expression.

We extract summary features (mean, std, quantiles of pixel intensity) at two spatial scales — the features at larger scales capture broader tissue context.

```
# Extract summary features at two scales
for scale in [1.0, 2.0]:
    feature_name = f"features_summary_scale{scale}"
    sq.im.calculate_image_features(
```

```

        adata,
        img.compute(),
        features='summary',
        key_added=feature_name,
        n_jobs=4,
        scale=scale,
    )

# Combine into one feature matrix
adata.obsm['features'] = pd.concat(
    [adata.obsm[f] for f in adata.obsm.keys() if 'features_summary' in f],
    axis='columns',
)
adata.obsm['features'].columns = ad.utils.make_index_unique(
    adata.obsm['features'].columns
)

print(f"Image features shape: {adata.obsm['features'].shape}")

```

```

100%                2688/2688 [00:36<00:00, 132.15/s]
100%                2688/2688 [01:24<00:00, 36.62/s]
Image features shape: (2688, 30)

```

5. Cluster Spots by Image Features

```

def cluster_features(features: pd.DataFrame, like=None) -> pd.Series:
    """Leiden clustering of image features."""
    if like is not None:
        features = features.filter(like=like)
    tmp_adata = ad.AnnData(features)
    sc.pp.scale(tmp_adata)
    sc.pp.pca(tmp_adata, n_comps=min(10, features.shape[1] - 1))
    sc.pp.neighbors(tmp_adata)
    sc.tl.leiden(tmp_adata, key_added='leiden_features')
    return tmp_adata.obs['leiden_features']

adata.obs['image_cluster'] = cluster_features(adata.obsm['features'])

print(f"Image-based clusters: {adata.obs['image_cluster'].nunique()}")

```

```

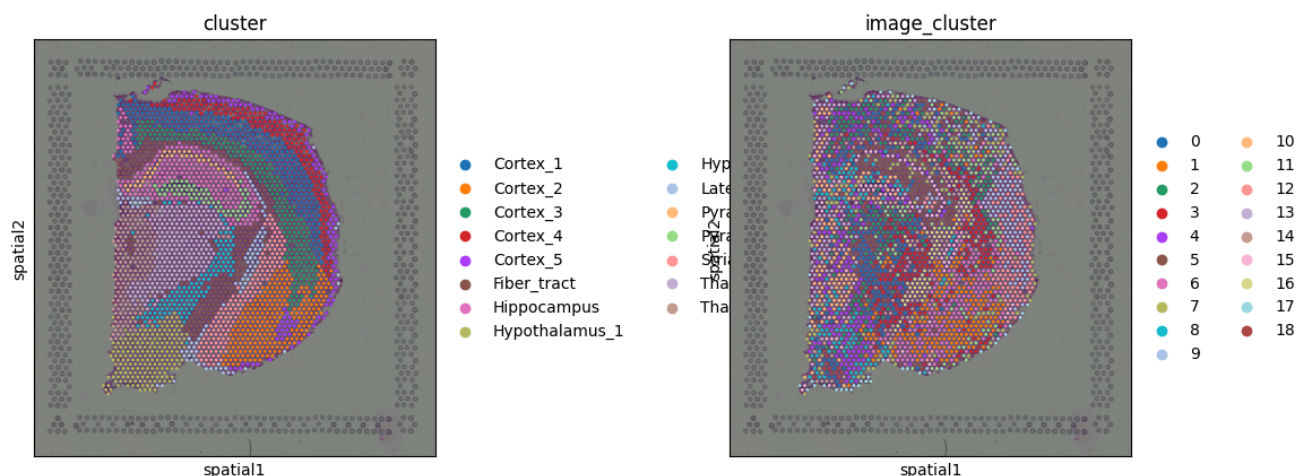
Image-based clusters: 19

```

```

sq.pl.spatial_scatter(
    adata,
    color=['cluster', 'image_cluster'],
    save='../results/figures/01_visium_hne/cluster_comparison.png'
)

```



6. Spatial Graph Analysis

6a. Build Spatial Neighbor Graph

```
sq.gr.spatial_neighbors(adata, coord_type='grid', n_neighs=6)
```

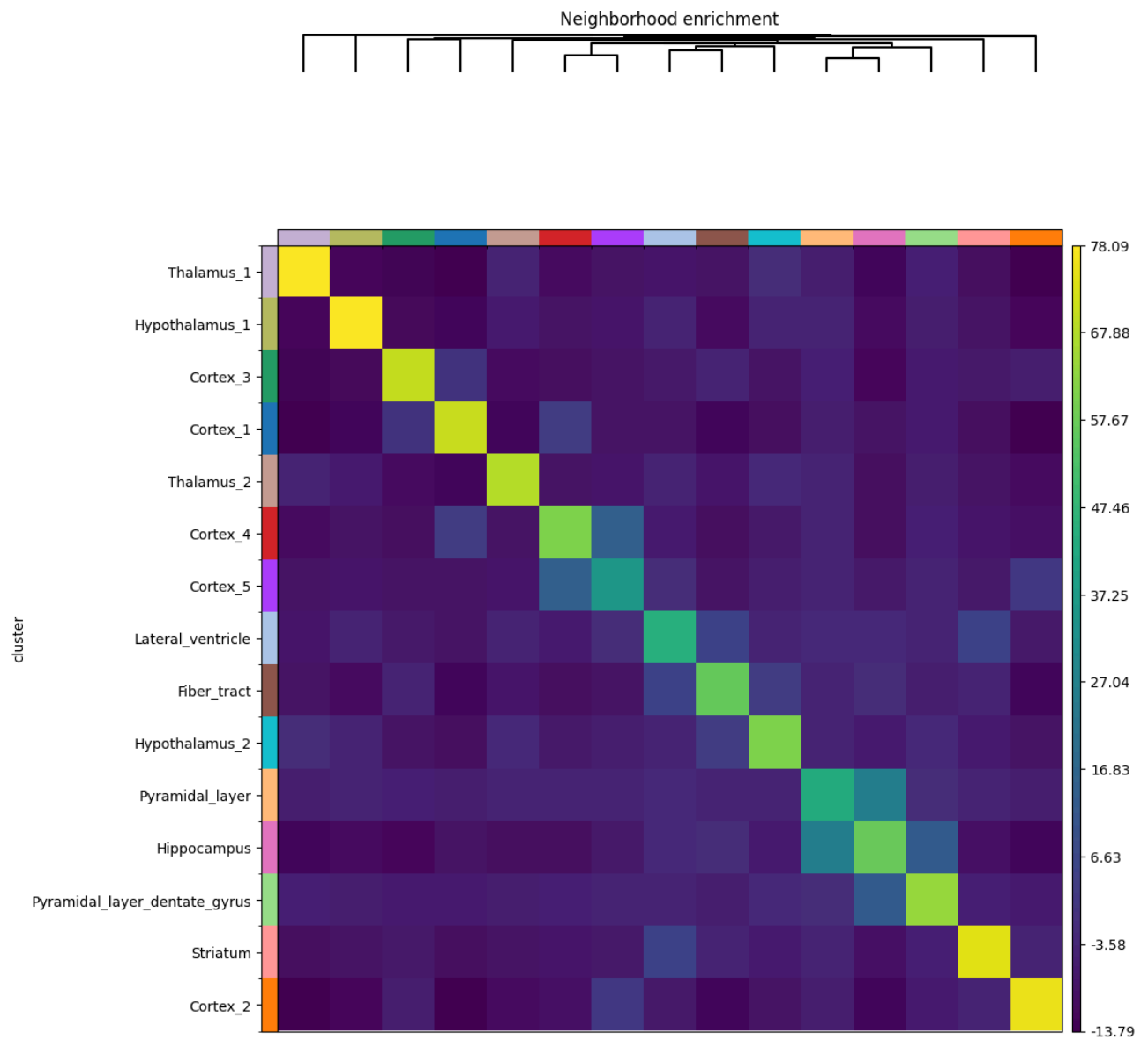
INFO Creating graph using `grid` coordinates and *None* transform and `1` libraries.

6b. Neighborhood Enrichment

```
sq.gr.nhood_enrichment(adata, cluster_key='cluster')
sq.pl.nhood_enrichment(
    adata,
    cluster_key='cluster',
    method='average',
    save='../results/figures/01_visium_hne/nhood_enrichment.png'
)
```

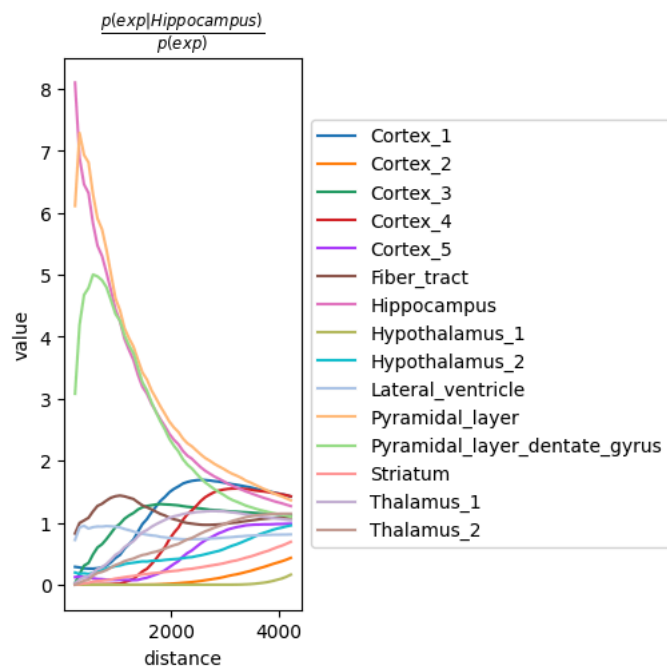
100%

1000/1000 [00:00<00:00, 1495.10/s]



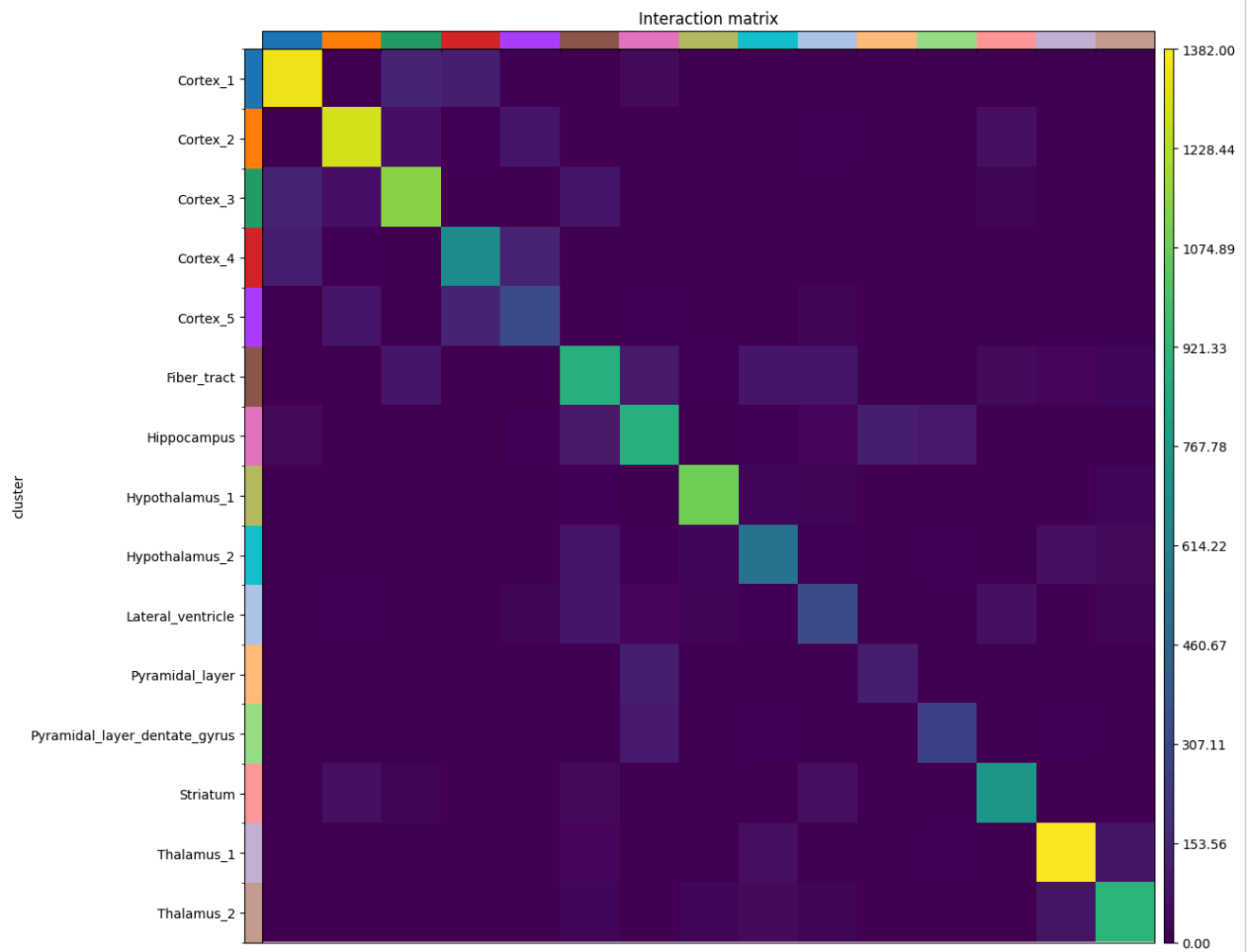
6c. Co-occurrence Score

```
sq.gr.co_occurrence(adata, cluster_key='cluster')
sq.pl.co_occurrence(
    adata,
    cluster_key='cluster',
    clusters='Hippocampus',
    save='../results/figures/01_visium_hne/co_occurrence.png'
)
```



6d. Interaction Matrix

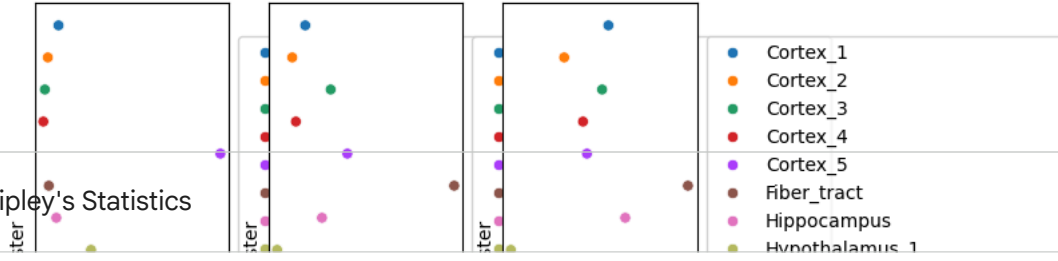
```
sq.gr.interaction_matrix(adata, cluster_key='cluster')
sq.pl.interaction_matrix(
    adata,
    cluster_key='cluster',
    save='../results/figures/01_visium_hne/interaction_matrix.png'
)
```



6e. Centrality Scores

```
sq.gr.centralities_scores(adata, cluster_key='cluster')
sq.pl.centralities_scores(
    adata,
    cluster_key='cluster',
    save='../results/figures/01_visium_hne/centralities_scores.png'
)
```

Average clustering Closeness centrality Degree centrality



6f. Ripley's Statistics

```
sq.gr.ripley(adata, cluster_key='cluster', mode='L')
sq.nl.ripley(
```